

Ransomware Over Modern Web Browsers: A Novel Strain and a New Defense Mechanism

HARUN OZ, Florida International University, Miami, United States

GÜLİZ SERAY TUNCAY, Google, California, United States

AHMET ARIS, ABBAS ACAR, LEONARDO BABUN, and SELCUK ULUAGAC, Florida International University, Miami, United States

Ransomware is an increasingly prevalent form of malware targeting end-users, governments, and businesses. As it has evolved, adversaries added new capabilities to their arsenal. We propose a next-generation browser-based ransomware, RøB, which performs its malicious actions via web technologies, File System Access API (FSA) and WebAssembly (Wasm). RøB uses this API through the victims' browsers; hence, it does not require the victims to download and install malicious binaries. We performed extensive evaluations with three different OSs, 23 file formats, 29 distinct directories, five cloud providers, and four antivirus solutions. Our evaluations show that RøB can encrypt various types of files in the local and cloud-integrated directories, external storage devices, and network-shared folders of victims. Our experiments also reveal that popular cloud solutions, Box Individual and Apple iCloud can be severely affected by RøB. Moreover, we conducted tests with commercial antivirus software such as AVG, Avast, Kaspersky, and Malware Bytes that perform sensitive directory and suspicious behavior monitoring against ransomware. We verified that RøB can evade these antivirus software and encrypt victim files. Moreover, existing ransomware detection solutions in the literature also cannot be a remedy against RøB due to its distinct features. Therefore, in this paper, we also propose RøBGUARD, a new detection system for RøB-like attacks. RøBGUARD monitors the web applications that use the FSA API via function hooking and uses a machine learning classifier to detect RøB-like attacks. We implemented a proof of concept version of RøBGUARD and our evaluation results show that RøBGUARD can detect RøB-like browser-based ransomware attacks effectively. We also provide future research directions that should be addressed in this domain.

CCS Concepts: • **Security and privacy** → **Malware and its mitigation**;

Additional Key Words and Phrases: Ransomware, malware, web security

ACM Reference Format:

Harun Oz, Güliz Seray Tuncay, Ahmet Aris, Abbas Acar, Leonardo Babun, and Selcuk Uluagac. 2025. Ransomware Over Modern Web Browsers: A Novel Strain and a New Defense Mechanism. *ACM Trans. Web* 19, 1, Article 6 (January 2025), 28 pages. <https://doi.org/10.1145/3708514>

This work was partially supported via US National Science Foundation's Intergovernmental Personnel Act Independent Research & Development Program, US National Security Agency (Award: H982302110324), Florida International University Graduate School, Cyber Florida, and Google ASPIRE Program. The views expressed are those of the authors only, not of the funding agencies.

Authors' Contact Information: Harun Oz, Ahmet Aris, Abbas Acar, Leonardo Babun, and Selcuk Uluagac, Florida International University, Miami, FL, USA; e-mail: hoz001@fiu.edu, aaris@fiu.edu, aacar001@fiu.edu, lbabu002@fiu.edu, suluagac@fiu.edu; Güliz Seray Tuncay, Google, CA, USA; e-mail: gulizseray@google.com.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1559-1131/2025/01-ART6

<https://doi.org/10.1145/3708514>

This work is an extended version of our earlier work published at the USENIX Security Symposium.¹

1 Introduction

Ransomware attacks have tremendously increased over the past years. This type of malware strain has been targeting not only individuals, but also governments, cities, and businesses in a variety of sectors. Recent victims include schools and universities, hospitals, water plants, oil and gas pipelines, meat processing plants, and even some police departments. Current figures predict that the cost of ransomware attacks will exceed \$10.5 trillion by 2025. These incidents precisely show that ransomware is one of the most crucial cyber threats at the moment.

Throughout the evolution of ransomware, adversaries have enhanced ransomware attacks not only by learning lessons from unsuccessful attacks but by also using different technologies. Nowadays, an enormous amount of new functionalities are being embedded into browsers and such technologies promise numerous advantages to users and web applications. Unfortunately, they can also extend the attack surface and inadvertently lead to users getting harmed. One such new technology is the **File System Access (FSA) Application Programming Interface (API)**, which has been published under the the **Web Platform Incubator Community Group (WICG)** [62] in order to enable web applications to interact with the users' local file systems. Similar to the permissions required by native applications when installing or accessing critical resources, the FSA API implements permission dialogs that are essential for mediating access between web applications and the user's local file system.

Although not on the W3C Standards Track at the moment, FSA API is embedded in and is fully supported by the most popular browsers like Chrome and Edge and is partially supported by Opera browser. The FSA API is already being used by the popular online development platform Microsoft Visual Studio Code, the online diagram editor draw.io and is becoming more and more popular. Even though this new technology can be used to develop powerful web applications, we found that it can also be abused by adversaries to develop a novel ransomware strain as a web application that encrypts the user's files from the browser. Such an attack would be performed by an adversary who designs a seemingly benign web application to trick the user to grant access to their sensitive portions of the local file system [48]. Afterward, the malicious application can encrypt the files and overwrite the original files with their encrypted versions and demand a ransom payment. As of now, we have found no evidence of any abuse of the FSA API in the wild, nor have we encountered any literature presenting it as a possible attack vector that the adversaries can utilize to perform browser-based ransomware attacks.

In this work, we design, implement, and analyze the impact of a novel, next generation, platform-independent, browser-based ransomware named RøB² that performs its malicious actions via web technologies, the FSA API, and **WebAssembly (Wasm)**. Unlike conventional ransomware, it does not require its victims to download or install any malicious binaries to their systems. It utilizes the FSA API through web browsers to encrypt the victim files. We implemented and comprehensively analyzed the impact of RøB. Our extensive analysis with three different OSs, 23 file formats, 29 distinct directories, five cloud providers, and four antivirus solutions show that RøB is capable of encrypting numerous types of files that can possess users' sensitive information. In addition, although the security model of the FSA API restricts access to some of the critical system directories (e.g., file system root, user's home, operating system), our experiments reveal that it can encrypt

¹Harun Oz, Ahmet Aris, Abbas Acar, Güliz Seray Tuncay, Leonardo Babun, and Selcuk Uluagac. RøB: Ransomware over Modern Web Browsers. In 32nd USENIX Security Symposium [45].

²RøB stands for **R**ansomware **o**n **B**rowsers. The letter o is replaced with ø for stylistic purposes.

files in user directories, data partitions (i.e., D:/), external storage devices such as external HDD and flash drives, shared network volumes, and cloud-integrated directories. Furthermore, we found that popular cloud solutions such as Apple iCloud and Box Individual can be severely affected by RøB, which can pose a severe security risk as these platforms have millions of users.

Several defense solutions exist against ransomware. There are many antivirus software that detect ransomware via monitoring the sensitive folders and suspicious behaviors on the victim's system. We performed analysis with full versions of antivirus software such as AVG, Kaspersky, Avast, and Malware Bytes and found that RøB can evade all of these antivirus solutions. In addition to antivirus software, many highly accurate ransomware detection studies exist in the literature that use static analysis features, or dynamic analysis features. However, they unfortunately fall short in detecting RøB due to its distinct features such as not requiring any installation, running on the browser, as well as using Wasm-based encryption libraries. Therefore, in this paper, we also implemented RøBGUARD, a defense solution which can detect RøB-like browser-based ransomware attacks. RøBGUARD hooks into the functions of the FSA API and monitors the operations of web applications. If a web application displays patterns of RøB-like ransomware attacks, it pauses the process of the web application and efficiently detects the attack using the **machine learning (ML)** classifiers with no file loss. We implemented and evaluated the performance of RøBGUARD considering both RøB-like malicious web applications and benign web applications that use the FSA API. Our experiments show that RøBGUARD can detect RøB-like ransomware attacks efficiently.

Contributions: The main contributions of this work are as follows:

- We implemented and thoroughly analyzed RøB, a novel platform-independent ransomware attack that performs its activities using web technologies such as the FSA API and Wasm.
- We evaluated the performance of RøB and conducted a comprehensive impact analysis on three different OSs, 29 distinct directories together with their subdirectories, five cloud providers, and five antivirus solutions.
- We evaluated the effectiveness of state-of-the-art existing ransomware detection solutions and found that they, unfortunately, fall short in detecting RøB-like ransomware due to its distinct features and highlighted the challenges that need to be addressed while designing a defense solution against a browser-based ransomware attack.
- We present the distinct features of RøB that differentiate it from conventional ransomware attacks. Based on these features, we propose RøBGUARD, a novel platform-independent defense solution that detects RøB-like browser-based ransomware attacks effectively.

Responsible Disclosure. The ransomware risk through the FSA API had been very briefly mentioned in the documentation [61]. However, we argue that the documentation, in its current form, significantly downplays the extent of the ransomware threat and gives misleading explanations regarding the efficacy of the countermeasures provided by the API. To share our findings, improve the security documentation, and contribute to the production of a working countermeasure against this threat, we contacted the editors of the FSA specification and the developers of the FSA API in the Chromium project at Google, through several channels. We submitted a security bug to Chromium outlining the above points in detail. We also had a video meeting and email exchanges with the developers of the FSA API. They acknowledged the issues and agreed to make changes in the security documentation to better explain the extent of the ransomware threat based on the findings of our work. In addition, we also responsibly disclosed the issue to cloud providers (e.g., Apple, Box) whose products we identified as being at risk and AV vendors. Apple reviewed the bug report and did not take responsibility as their product was indirectly impacted by this issue. We have yet to receive any responses from the other parties at the time of this writing.

Ethics. Due to ethical considerations, we did not make the RøB (i.e., ransomware) implementation publicly available. In addition, we performed all the analysis on local servers; hence, no human subjects have been involved in this research.

2 Background

2.1 Ransomware

Ransomware is a type of malware that limits users from accessing their systems until a ransom amount is paid. Since its first appearance, ransomware creators used different web technologies to develop more sophisticated ransomware strains. Ransomware can be classified as cryptographic or locker ransomware. Cryptographic ransomware encrypts victim files and demands a ransom payment for decryption. Locker ransomware locks system components to prevent a victim's access to the system and demands a ransom payment to unlock. Ransomware families use almost every infection technique such as malvertisement and social engineering.

2.2 The File System Access API

Overview. The Web Platform Incubator Community Group³ created the File System Access (FSA) API to enable the development of powerful web applications that interact with files that are located in users' local file systems [62]. This API can be used to develop various web applications that can interact with a user's files in a convenient way. For instance, developers can build online document editors, business tools, and **integrated development environments (IDE)** that can directly interact with a user's local file system from the browser without any installation. Moreover, this API can be used to implement/port the web versions of existing native applications such as AutoCAD. Although it is not on the Standards Track as of now, the FSA API is embedded in and fully supported by the most popular browsers like Chrome and Edge [58]. Moreover, Mozilla and Brave, did not integrate this API into their browsers due to some concerns [10, 40]. Particularly, Mozilla tagged this API as harmful, because they did not think "... meaningful end-user consent is possible to obtain ..." [40] and Brave considers this API as "non-standard privacy risking" [10]. Additionally, Safari's WebKit recently partially implemented the FSA API in its engine [1]. However, they restricted the capabilities of FSA API by allowing only access to the Origin Private File System, which is mapped to a database outside of the user's OS. Our findings in this paper align with the concerns of other major browsers.

In our work, we used Chrome 85 for our implementation and experiments.⁴ In this version, the FSA API has a single-entry point, named `chooseFileSystemEntries()`. This entry point opens a picker dialog allowing a user to select multiple files or directories. After the selection, the browser asks for the user's permission to read the contents of the files or directories via opening a read permission dialog. Depending on whether the user selects a file or a directory, the API returns either a `FileSystemFileHandle` or `FileSystemDirectoryHandle`. These handles provide methods to interact with the files or directories. When a web application calls the `createWritable()` method on a `FileSystemFileHandle` to modify a file, the browser prompts the write permission dialog for the user. Subsequent calls to `createWritable()` for the same file within the same session do not prompt the user for permissions again. Similarly, if the user grants both read and write permissions for a directory, the returned `FileSystemDirectoryHandle` gives access to all files and subdirectories within it. Analogous to the file handle, any modifications on the files in the same directory within the same session do not require repeated permission prompts.

³<https://wicg.io/>

⁴Chrome 85 (M85) was released in August 2020. The latest version of Chrome, as of October 2024, is M132.

Security Model. The security model of the FSA API considers a few attacks including the possibility of attacks that encrypt files (i.e., ransomware), malware storing, and execution [17]. There are two main strategies adopted by the API to tackle these attacks. First, it utilizes a permission model which requires web applications to obtain access from users via permission dialogues. This permission model is simple; there is one permission dialog for read access and another one for write access. However, the effectiveness of this model is limited as adversaries can hide their real intents and use social engineering to get a user's explicit permission. Second, the FSA API utilizes an access limitation strategy by blocking web applications' access to the critical parts of the file system such as the root directory as well as the user's home, **operating system (OS)**, and browser profile directories. However, this strategy also falls short in preventing the attack we present in this work as the sub-directories of these restricted directories and any directory that are not explicitly blocked by this approach (e.g., cloud directory) can still be encrypted. We analyze the effectiveness of this approach further in Section 4.4.

3 Motivation and Threat Model

3.1 Motivation

This work assumes that Mallory, an adversary, creates a benign-looking malicious web application or compromises a benign web application that can access the local file system of victims using the FSA API. Mallory's web application presents itself as an online tool that enables users to create featured Instagram stories using images, texts, and other content online for free. Alice visits Mallory's malicious website, which employs social engineering to hide its malicious purpose (i.e., run as ransomware on the visitor's web browser). Upon checking the website's design and fake user reviews, Alice is convinced of its legitimacy. She also verifies the website certificate information by clicking on the padlock icon in the browser's address bar. As malicious websites can use valid certificates, Mallory's website also uses one, further convincing Alice to trust the website. To create Instagram stories, Alice selects directories on her file system to allow Mallory's web application to view and modify the files, thus enabling the ransomware to encrypt the files and demand a ransom. Although the API limits access to system directories, the root directory of the file system, and the user's home and browser profile directories, the malicious web application still accesses and modifies other significant parts of Alice's local file system.

3.2 Threat Model

In our threat model, we consider the scenarios where an attacker creates a malicious web application or compromises an existing web application and then tricks the user into granting read/write access, or the attacker compromises a site that already has access to some files and abuses it. Specifically, the attacker uses social engineering tactics to lure its victims to the web application and then tricks them into granting read and write access through seemingly benign interactions. Afterward, the adversary can use any encryption algorithm to encrypt the victims' files and overwrite the local files. For encryption, the adversary might utilize an encryption library in Wasm or JavaScript, or implement the encryption algorithms independently. Subsequently, the adversary may employ various extortion methods, such as demanding cryptocurrencies, similar to those used by traditional ransomware families, to secure payment. It is crucial to note that in this threat model, the FSA API functions as intended but is exploited by the attacker to access and overwrite the user's local files.

Attack Practicality. As described above, for browser-based ransomware to be effective, the user needs to navigate to a malicious site and grant read/write access. This threat model can be considered practical for several reasons. Firstly, social engineering techniques are still among the leading cybercrimes perpetrated by attackers in the wild [2, 59], affecting large enterprises and end users.

Secondly, the threat model involving a user accessing a malicious URL [16] and a tricked user granting permission [3] has been utilized in other state-of-the-art studies. With advances in browsers and the trend of porting native applications to the web, attackers may develop more compelling strategies to successfully lure users into using their ransomware. Additionally, the average layman might find the browser more trustworthy and grant read/write requests on file systems more inconsiderately than they would download, install, and execute unknown software [46, 60]. Furthermore, the lack of clear warnings in the current UI about potential ransomware threats significantly increases the risk of users succumbing to these attacks. Last but not least, while traditional ransomware must bypass many built-in checkpoints (e.g., email attachment scanners, download scanners, local antivirus programs) throughout the attack, RøB-like ransomware can reach the victim directly without bypassing these steps, which would also increase the practicality of the threat model used in this chapter. Although we described the example scenario, there can be several other practical scenarios. As more desktop applications move to the web, practical scenarios will include, but are not limited to:

- *Online Client-Side Virus Scanner*: Mallory's web application can offer to perform virus scanning on demand for free without being installed on the system. Since the majority of the existing antivirus software is not free and has high resource overheads, Alice can opt to try the online virus scanner.
- *Online Client-Side Media Editor*: Mallory's web application can offer to enable users to edit media files (e.g., photo, audio, video) and create free photo books and albums without being installed on the system. Alice can try to use Mallory's web application with various motivations such as being free or not requiring installation. Alice can be an employee and may need admin rights to install software on her workstation. She can consider overcoming such limitations by using an online tool.
- *Online Client-Side File Optimizer*: Mallory's web application can offer to enable users to optimize the size of the images, audio, video and document files of the users and save disk space without losing quality free of charge.

4 RøB - A New Ransomware Strain

RøB is a browser-based ransomware that performs its malicious actions by utilizing web technologies such as the FSA API and Wasm. In this section, we give the details regarding the design, implementation, and impact evaluation of our proof-of-concept ransomware implementation RøB.

4.1 System Model

Figure 1 shows the architecture system model of the RøB that includes five modules: *Backend*, *Web User Interface (UI)*, *File System Access*, *Encryption*, and *Extortion*. The first component of RøB is the Backend module that stores all other components of RøB in a hosting environment such as the cloud. When a victim starts to interact with the web application, the Backend module creates the keys and the session ID for the client (victim) and serves the RøB components to them. The second module is a Web UI that is designed by the attacker as a seemingly benign web application to fool the victim. The File System Access module contains the FSA API functions to access the victim's files and modify them effectively. The Encryption module includes the functions and modules to encrypt the victims' files successfully. Finally, the Extortion component informs the victims about the attack and includes information for the ransom payment.

4.2 Implementation of RøB

We implemented RøB as a web application that works on the Chromium-based browsers. At a glance, the implementation of RøB uses the **File System Access (FSA)** to access files and uses

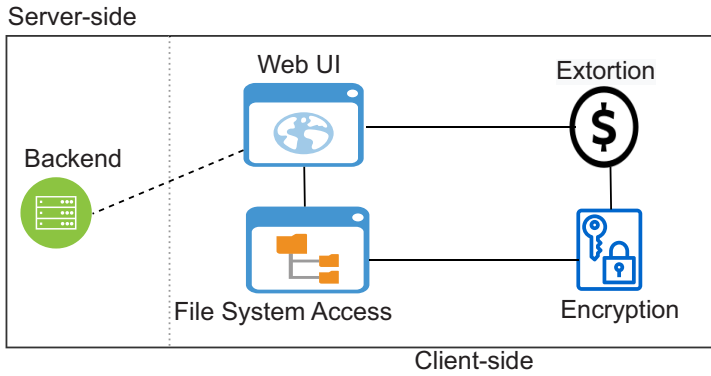


Fig. 1. System model of RøB ransomware.

hybrid encryption. Specifically, it encrypts all of the victim files with 256-bit **Advanced Standard Encryption (AES)** and encrypts the AES symmetric key with 2048-bit RSA. It utilizes AES and RSA implementations in both Wasm and JavaScript. For proof-of-concept implementation purposes, we focused on implementing the client-side modules of RøB (i.e., Web UI, File System Access, Encryption, and Extortion modules). The implementation of RøB uses the FSA API to access the files of victims. We utilized hybrid encryption so that the recovery of the encrypted files is not possible. To be more specific, RøB encrypts all of the victim files in a read-encrypt-overwrite loop with 256-bit AES and finally encrypts the AES symmetric key with 2048-bit RSA public key obtained from the Backend module of the adversary so that the recovery of the encrypted files is not possible. We implemented the client-side modules of RøB as a proof-of-concept.

Backend Module. This module receives HTTP requests from clients (victims), it creates a public-private key pair and a unique ID for each victim. Keys and victim IDs are stored in its database. Afterward, it sends an HTTP response to the victim, which includes the other components of RøB, client ID, and the generated encryption keys for the client. The keys stored by the Backend module for each client are shared with the ones who make payments for the recovery of their files.

Web User Interface Module. This module includes the contents regarding the look of the website that aims to trick victims to enable RøB to access their local file system. The attacker can design the Web UI component differently depending on the malicious scenario. For example, this module can be designed by the adversary as a media (e.g., picture, video) editor.

File System Access Module. File System Access Module Implementation. This module uses the FSA API to access and modify the victim's files as outlined in Algorithm 1. In Line 1, `showOpenFilePicker()` function is invoked which opens a directory select dialogue for the user. When the user selects a directory and grants a read permission, the API returns a `FileSystemDirectoryHandle` that contains the methods needed to interact with the files of the user. The statements in Lines 2-8, iterate through files in the directory selected by the user. `getFile()` method called in Line 3, returns a `FileSystemEntry` object of a file and forwards it to `encrypt()` which returns the encrypted file contents. Afterward, `createWritable()` in Line 5, asks for the user's permission to modify the file. Once the permission to write has been granted by a user, the `write()` method overwrites the file. Thus, RøB reads, encrypts, and overwrites the encrypted content for every file in the selected directory of the user one by one. Since the API does not need users to grant explicit permissions for each file within the directory (as read and write permissions are obtained for the directory), RøB can continue performing its malicious actions without prompting the users as long as the session continues.

ALGORITHM 1: Algorithm of RøB.

```

1  dirHandle = window.showDirectoryPicker();
2  for entry  $\in$  dirHandle.values() do
3      file = entry.getFile();
4      encryptedContent = encrypt(file);
5      writable = file.createWritable();
6      writable.write(encryptedContent);
7      writable.close();
8  end

```

Encryption Module. RøB uses a hybrid encryption technique to encrypt the victim's files. Particularly, the Encryption module first generates a symmetric key and encrypts the victim's files with AES256 algorithm. After the encryption of all of the files, it encrypts the AES key with RSA-2048 using the public key that is generated by the Backend module when the victim initially made a request to the malicious web application. We implemented two versions of the Encryption module to assess the performance and overhead of different implementation options. In the first version, we implement the Encryption module using the Enigma library which uses a Wasm-compiled version of OpenSSL to increase the performance. In the second version, we used the WebCrypto API which utilizes only JavaScript to perform encryption operations. In both implementations the encrypt() function takes file object as an input and returns its encrypted contents. We would like to note that, adversaries can employ JavaScript or Wasm implementations of other known encryption algorithms such as the **Elliptic Curve Cryptography (ECC)**, or their own encryption algorithms. However, for the evaluation purposes, we only employed the aforementioned encryption algorithms and the libraries that implement such algorithms.

Extortion Module. When the encryption process is finished, this module redirects the user to the ransom note link that informs the victim about the ransomware attack and gives details regarding the ransom payment method. RøB can employ Bitcoin as the payment method and can also use secure email services such as Proton that are used by the malicious actors. In our implementation, this module redirects the victim to another webpage that displays the victim ID assigned by the Backend module for the victim, as well as the ransomware note and the associated payment details.

4.3 Performance Analysis of RøB

In this subsection we analyze the performance of RøB with two different Encryption module implementations: Enigma library (for Wasm technology) and WebCrypto library (for JavaScript technology). Particularly, we analyze computation time (i.e., the time it takes to encrypt files of the victim) and overhead (i.e., CPU and RAM usage) of RøB. The analysis is performed and evaluated on a system running macOS Big Sur 11.0.1 with an 2,7 GHz Dual-Core Intel Core i5 processor and 8GB of available RAM.

4.3.1 Computation Time Analysis. To analyze the computation time of RøB, we run RøB implementation on Chrome and we used Chrome DevTools to measure the computation time performance of it. Although RøB can be configured to select specific file types to encrypt, it obtains files as a stream of data through the FSA API. Therefore, we give the computation time analysis of RøB independent of the file types. We performed the computation time analysis of RøB by encrypting files with various sizes of 1MB, 10MB, and 100MB. The computation time analysis of two RøB implementations with respect to varying file sizes. To be more specific, RøB can encrypt victim files of 1MB in 1612.96 (ms), 10MB in 2593.72 (ms), and 100MB in 3010.74 (ms). Considering the

Wasm and JavaScript implementations of the Encryption module, we see that, since the loading process of the Wasm binaries to the browser presents a small overhead, RøB with JavaScript is more efficient than RøB with Wasm for files sizes smaller than 10MB. However, as the file sizes get bigger, RøB with Wasm starts to outperform the RøB with JavaScript.

4.3.2 Overhead Analysis. On average, RøB with JavaScript and RøB with Wasm used 0.1% of available RAM. During the encryption process, RøB with JavaScript used maximum of 0.120MB of RAM and RøB with Wasm utilized 0.125MB of available RAM. In terms of the CPU usage, on average, RøB with Wasm used 15% of the CPU during the attack. During the encryption process, CPU usage of RøB with Wasm peaked to 25%-40%. On the other hand, RøB with JavaScript used 10% of the average CPU and this implementation utilized 25%-35% of available CPU during the encryption phase. So, both implementations of RøB used between 25%-40% of CPU for 1-2 seconds.

4.3.3 Findings. RøB encrypts the victim's sensitive files with a hybrid encryption to combine the benefits of asymmetric encryption and symmetric encryption. Hybrid encryption is used by several ransomware strains in the wild to quickly encrypt files using a symmetric key encryption (e.g., AES) and protect the symmetric key with an asymmetric key encryption (e.g., RSA). Since AES and RSA are the standard encryption algorithms used by almost every entity on the Internet for security, the only option for the victim to recover his/her files will be to pay the ransom amount dictated by the adversary in RøB. Furthermore, the computation time and overhead analysis show that even though RøB uses web technologies to perform its actions, it quickly and securely encrypts all types of victim files. Therefore, the victim will not have time to respond to the attack. Furthermore, given to its reasonable amount of RAM and CPU usage during the attack, it does not cause latency in the browser. So, RøB even completes the attack while the victim is using another web application in the same browser. Nevertheless, the different implementations of the RøB may have their own advantages and disadvantages. For example, RøB with JavaScript is faster than the RøB with Wasm on the encrypting file sizes smaller than 10MB. On the other hand, RøB with Wasm implementation is faster in the case of encrypting large file sizes. Therefore, depending on the targeted victim type (adversaries can embed a list of domains to their ransomware which is used to identify the victim type) the adversary can prefer a specific implementation of RøB. For example, if the attacker intends to encrypt a single directory of the victim with small file sizes, utilization of JavaScript to encrypt the files is more suitable. On the other hand, if the attacker targets the employee of a private company, the targeted directory may include a large number of business documents. So, in this type of attack RøB with Wasm is more suitable. Given the popularity of JavaScript, the cryptographic libraries implemented with JavaScript are small and easy to use.

4.4 Attack Surface Investigation

In this subsection, we perform experiments to study the impact of RøB on victim files and directories.

Evaluation Setup. We performed our first analysis on a computer (*Test-PC:1*) running Windows 10 Pro with a 2,6 GHz Intel Core i7 processor and 16GB of available RAM. We also employed computers powered by Linux (*Test-PC:2* - Ubuntu 20.04 LTS with a 2,6 GHz Intel Core i7 processor and 16GB of RAM), and macOS (*Test-PC:3* - macOS Big Sur 11.0.1 with a 2,7 GHz Dual-Core Intel Core i5 processor and 8GB of RAM) for our evaluations. This enabled us to understand the type of directories affected in multiple platforms used by a diverse set of users. Moreover, the analysis with network-shared folders required two Windows computers to share a folder over the network. Therefore, in addition to the *Test-PC:1*, we employed a second Windows-based computer (*Test-PC:4*) with the same specs of *Test-PC:1*.

Table 1. Local Directories Affected by RøB

Directory	Windows		Linux		macOS	
	FDA	SDA	FDA	SDA	FDA	SDA
Documents	Denied	Yes	Denied	Yes	Denied	Yes
Desktop	Denied	Yes	Denied	Yes	Denied	Yes
Pictures	Yes	Yes	Yes	Yes	Yes	Yes
Videos	Yes	Yes	Yes	Yes	Yes	Yes
Music	Yes	Yes	Yes	Yes	Yes	Yes
Downloads	Denied	Yes	Denied	Yes	Denied	Yes
Data Partition	Denied	Yes	Yes	Yes	Yes	Yes

FDA: Full Directory Access, SDA: Subdirectory Access. "Yes" indicates FDA/SDA, "Denied" means access is denied.

Types of Directories Affected. In this analysis, we investigate and identify the affected directories in various operating systems (e.g., Windows, Linux, and macOS) that RøB can encrypt. We analyze the access limitation of the FSA API on a set of directories such as local directories, cloud-integrated directories, external storage devices, and network shared folders and determine if RøB can access and encrypt full directory contents or only contents of subdirectories.

- *Local Directories.* To determine the directories that are affected by RøB, we created test folders that include 50 files with different file types docx, xlsx, pdf, txt, jpeg on test computers *Test-PC:1*, *Test-PC:2*, and *Test-PC:3*. Subsequently, we placed the test directories in different parts of the file systems (in total 21 different directories on three OSs) of the test computers. On each test computer, we ran RøB as a web application using a Node.js server, browsed the RøB web application, and tried to select the directory under test that has our folder with the files.

As explained earlier, the security model of the FSA API prevents the web application from accessing specific directories. We tested those directories (i.e., the root directory of the file system, user's home, OS, and browser profile directories) on *Test-PC:1* and verified that the FSA API does not allow a user to select those directories. This implies that the security model of the FSA API prevents RøB from accessing these directories. Nevertheless, we continued our tests by placing our test directories in other parts of the local file system to choose such directories via RøB. We realized that RøB can access and encrypt the full contents (including subdirectories) of the Windows directories of Pictures, Videos, and Music. In addition, we found out that although the security model of the FSA API does not allow a user to choose the Documents, Desktop, Downloads, and the data partition directories (e.g., D:/), there is no access limitation dictated by the API on their subdirectories.

To summarize, our evaluation showed that RøB can arbitrarily encrypt the full contents of Pictures, Videos, and Music, and the subdirectories of Documents, Desktop, Downloads, and data partition directories on a Windows computer. Considering the other test computers (i.e., *Test-PC:2* and *Test-PC:3* running Linux and macOS respectively), our evaluation showed that the FSA API has very similar access limitations for web applications (hence for RøB) on computers powered by these operating systems. Table 1 provides a summary of our findings. Once RøB can have full directory access or subdirectory access, it encrypts all of the files inside. Although we see similar full and subdirectory access patterns of RøB for most of the local directories of Windows, Linux, and macOS, we see an interesting pattern for data partitions. While the FSA API prevents RøB from fully accessing the data partition and allows only subdirectories to be accessed for Windows, it allows full access to data partition and subdirectories in both Linux and macOS platforms.

- *Cloud-Integrated Directories.* To analyze the effects of RøB on the files that are stored in cloud-integrated directories, we first created dummy accounts on popular cloud platforms such as Google

Drive, DropBox, Box, iCloud, and Microsoft One Drive. These platforms have desktop applications that allow users to work on the files locally and sync them to the cloud using file system integration. We downloaded their desktop applications to *Test-PC:1* to enable that feature and create their integrated directories. Subsequently, we placed 50 files with different file types in each cloud-integrated directories on *Test-PC:1* and chose these directories as the target while running RøB.

Our experiments show that when RøB accessed these directories, it successfully encrypted the files in those cloud-integrated directories. After that, the sync engine of the cloud provider reflected the modifications to the cloud storage and the changes made to the files in the cloud became persistent.

We note that even though these platforms have their built-in ransomware detection mechanisms, none of the cloud platforms were able to detect RøB during or after the attack. On the other hand, as a ransomware protection method, some cloud platforms use versioning techniques [23]. If the files that are stored in the cloud are attacked by ransomware, the cloud user can retrieve the earlier (unencrypted, original) version of the files. However, this feature is not standard among all of the cloud providers and every cloud vendor has its own versioning scheme. While cloud providers such as Dropbox keep track of versions of files for basic, plus, and family users for 30 days and professional and business users for 180 days [20], OneDrive of Microsoft keeps 25 versions of the stored files [43]. Differently, Google Drive can keep versions of files for 30 days or the first 100 versions of them [23]. Box has different solutions with various versioning features. Specifically, while Box Individual keeps only one version of a file, Personal Pro and Business Starter solutions keep 10 and 25 file versions, respectively. On the other hand, iCloud does not utilize the versioning feature. We further investigated the versioning scheme of the cloud providers and realized that Dropbox, Google Drive, and Microsoft OneDrive can store more than 100 versions of the files and RøB cannot have a permanent effect on the files stored by them. In addition, Box Individual and Apple iCloud do not have a versioning scheme and we verified that the files encrypted by RøB cannot be recovered by such solutions.

We conclude that if victims are using iCloud or Box Individual, RøB can cause them to lose their significant files unless a ransom is paid. RøB-like ransomware attacks can create significant damage to iCloud users. In addition, although the versioning strategy employed by Dropbox, Google Drive, and One Drive seems to be resilient against RøB, it should not be considered as a silver bullet. This is due to the fact that the backup files in the version history do not always reflect the most recent state of a file. If RøB is launched before a backup is done, then critical changes on user files can still be lost.

- *External Storage Devices.* To test the impact of RøB on external disks, we placed a test directory that included 50 files with different file types in a test external disk (e.g., Western Digital 4TB) and a test flash drive (e.g., Toshiba 16GB) connected to *Test-PC:1*. After, we ran RøB and chose the directory in each external storage device as the target.

Our experiments showed that RøB is able to encrypt all of the files located in the selected directories in each external storage device. As external storage devices are used by ordinary users and enterprise users to backup important data, RøB can have detrimental effects on the files stored in such devices. Unfortunately, such devices are outside the scope of the security model of the FSA API which leaves them prone to RøB-like ransomware attacks.

- *Network Shared Folders.* To test the effect of RøB on network shared folders, we created a test directory that included 50 files with different file types in our test computer *Test-PC:1* and we shared it over the network with *Test-PC:4*. After that, we run RøB on *Test-PC:4* and selected the test directory that is shared over the network as a target.

- *Results and Findings.* We observed that RøB is able to encrypt the files that reside in the shared folder which shows that running RøB in one computer can affect the folders/files that are shared

by multiple computers over the network. Shared folders are frequently used by both individuals and enterprises and if sensitive/important files are stored in the shared folder, the effects of RøB on these folders can be very serious.

4.5 Desktop vs. Browser-based Ransomware

In this section, we discuss the fundamental differences between browser-based ransomware and desktop ransomware.

Infection Strategies. Browser-based and desktop ransomware differ significantly in their approaches to infecting target systems. Browser-based ransomware, like RøB, can be designed as a benign-looking web application or compromises a benign application and encrypts the victim files through the browser. In contrast, desktop ransomware families are designed as binary applications and often infect the victim system by using traditional malware infection techniques such as drive-by-download [47].

Initial User Access. Desktop ransomware typically spreads via advertisements, or emails, which are used to trigger downloading the malicious payload. In comparison, browser-based ransomware, such as RøB, is a malicious web application that needs to attract its victims to its domain in some way. Similar social engineering methods RøB be used by RøB to distribute the link and to gain initial user access. To attract more users, it can also be designed as a benign-looking web application (e.g., a free media editor).

Infection and Execution. For desktop ransomware to be effective, the user (victim) must download (i.e., infect) and execute the binary on its system. Unlike desktop ransomware, RøB is fileless, i.e., no download or execution is required. However, after luring victims to its domain, browser-based ransomware still needs to trick its victims into granting read and write access. In this manner, browser-based ransomware has the advantage of executing its actions via browsers, which are used by millions of users and perceived as trustworthy while desktop ransomware requires users to download and execute unknown binaries. Moreover, it executes its actions through browsers, which are used by millions of users and considered trustworthy software while desktop-based ransomware requires users to download and execute unknown binaries.

Encryption. RøB uses Wasm to encrypt the victim files. As it employs all of the encryption logic in the Wasm, unlike desktop ransomware, it does not utilize the platform's encryption libraries or OS system calls. Hence, it can hide its file encryption process from the defense solutions that monitor the system calls made to the platform's encryption APIs.

Extortion. Desktop ransomware delivers ransom payment information to the victim via leaving a ransom note on the desktop, changing the desktop background, or using lock screens. However, RøB cannot employ those techniques and has to find other ways. Particularly, RøB can: (A) redirect the victim to the extortion page upon finishing encryption, (B) add ransom note files to the parent folders of the encrypted files, and (C) change the names of encrypted files and add ransom note to file names by adhering to maximum path length constraints of platforms. If Option A is employed, then the ransom note may never reach the victim if the victim closes the page before the encryption is completed. Options B and C are better ones, ensuring that the ransom note will be reachable to the victim even if the page is closed. However, those options may give clues to defense solutions in detecting RøB.

5 Effectiveness of Current Defenses

In this section, we investigate the effectiveness of existing ransomware defense solutions against RøB. First, we explain the characteristics of RøB, which create new challenges for existing defense solutions. Second, we share the results of our experiments with the premium versions of popular

antivirus solutions in detecting RøB. Last, we review the existing ransomware detection systems in the literature and explain the need for a new defense system against RøB.

5.1 Antivirus Solutions

In this section, we analyze the effectiveness of popular antivirus solutions against RøB. Specifically, we tested the effectiveness of the full versions of five different antivirus solutions, namely Malware Bytes, AVG Antivirus, Kaspersky, Trend Micro, and Avast against RøB. We chose these antivirus solutions because they explicitly promise ransomware defense via malicious behavior monitoring for users. - *Evaluation Methodology.* To perform our experiments, we downloaded and installed each antivirus solution to our test computer *Test-PC:1*, put test directories with 10, 50, and 100 files with various file types, and checked if they can detect RøB when it is instructed to run on the test folder. In other words, we performed three sets of tests where our test folder had 10 files in the first test, 50 files in the second test, and 100 files in the third test for each antivirus solution.

- *Malware Bytes.* Malware Bytes Premium [33] promises to protect users' documents and financial files against ransomware. We installed Malware Bytes Premium and selected the test directory when running RøB. We verified that Malware Bytes Premium could not detect RøB. Moreover, we also tried the browser extension of Malware Bytes, namely Browser Guard [32] which blocks malicious web pages and web applications that include ransomware. We performed another test while Malware Bytes's Browser Guard extension is activated and it could not detect RøB.
- *AVG Antivirus.* We installed the full version of the AVG Internet Security [8], which promises ransomware protection. AVG Internet Security monitors sensitive folders such as Documents, Pictures, and the like, and allows a user to add a folder to the sensitive folders list for monitoring. We added the path of the test directory to the sensitive folders list. Following that, we ran RøB and chose the test directory as the target. AVG Internet Security was not able to detect RøB.
- *Kaspersky.* We installed the full version of Kaspersky Total Security [27]. Kaspersky Total Security has an anti-ransomware tool that monitors the personal computer for ransomware-like behavior in real time. Nevertheless, we ran RøB and could successfully encrypt the files within the test folder. Kaspersky Total Security was not able to detect RøB in any of the test cases.
- *Trend Micro.* We installed the full version of Trend Micro Antivirus+ Security [57], which promises to utilize prevent various threats such as ransomware and online attacks via its Advanced AI Learning utility. We performed three tests with 10, 50, and 100 files in the test folder. We confirmed that Trend Micro Antivirus+ Security was not able to detect RøB in any of our test experiments.
- *Avast.* We installed the full version of Avast One Essential [7], which promises to provide ransomware protection via monitoring important folders. We added the path of the folder among the important files/folders to be monitored. We verified that Avast One Essential was not able to detect RøB in any of our test cases.

Our findings show that as browser-based ransomware has not yet been seen in the wild, traditional antivirus solutions might be more geared towards detecting threats in files and applications rather than those originating from browser-based ransomware. Moreover, our study focused on consumer-grade antivirus solutions, which may not have the same level of robustness or comprehensive features as enterprise-grade antivirus systems. This distinction can significantly influence the effectiveness of ransomware detection and should be carefully considered when interpreting the results of our experiments. Thus, while our findings highlight potential limitation of popular antivirus products, they also underline the need for a nuanced understanding of different antivirus capabilities in both consumer and enterprise contexts.

5.2 State-of-the-art Ransomware Defense

The ransomware defense approaches for PCs can be grouped into three categories: (1) Static analysis-based detection methods, (2) Dynamic analysis-based detection methods, and (3) Key extraction-based recovery solutions.

Static Analysis-based Solutions. Many researchers proposed static analysis-based solutions [35, 64] that utilize structural features such as strings and opcodes to detect ransomware. Although those solutions can detect well-known ransomware strains, they are vulnerable to common evasion attempts such as obfuscation [9, 49]. In the concept of browser-based ransomware attacks, the adversaries are free to use any tool available and employ obfuscation techniques to evade all types of static analysis-based tools. Therefore, such solutions are not suitable for browser-based ransomware attacks.

Dynamic Analysis-based Solutions. The dynamic analysis-based solutions use behavioral features such as network activity, API/system calls, I/O access patterns, and file system activity to detect ransomware [47]. First, RøB does not need frequent C&C server communication. In fact, only one HTTP request made to the Backend module of RøB is sufficient for it to be sent in an HTTP response packet and perform its malicious actions. In addition, RøB's communication is based on HTTP over TCP which is used by almost every benign website and web application. Therefore, the solutions that use network traffic features [12, 13, 39] would struggle to detect RøB. Due to the high computation cost of the malware analysis environments [44], the ransomware analysis environments such as [18, 28] (albeit was very useful) has become impractical against RøB-like attacks as it is not practical to analyze every website before the visit of a user in such analysis environments. Additionally, ransomware defense solutions such as [18, 29, 55] utilize the features retrieved from the file system activities such as folder listing, files written, read, renamed, and deleted. These defense solutions have been designed by monitoring the file system activities performed by the process of the ransomware executable. Nevertheless, the file system activities of RøB are different from the traditional ransomware defense solution. So, these solutions will not be effective to detect RøB.

Moreover, RøB uses browsers to perform its malicious actions. Running on the browser without being installed on a system can create additional challenges for API/system call monitoring solutions [4, 30, 34, 65]. Considering RøB, such solutions face two difficulties: (1) In RøB, the adversary is free to embed his own encryption code to the Encryption module which will not use the crypto APIs of the OS. (2) RøB runs in the browser (a benign program) and monitoring the system calls made by the browser will introduce an overhead, as each website visit creates many browser processes and users can have multiple tabs open. Therefore, monitoring API/system calls of browser processes would incur high overhead. For these reasons, API/system calls monitoring solutions will not be practical and effective in detecting RøB.

Key Extraction-based Solutions. Some ransomware defense systems use memory forensic techniques to retrieve the keys of the attacker to recover the files. The study in [30] presents a ransomware recovery mechanism that stores encryption keys by hooking the crypto functions of the OS. Similarly, the study in [36] combines process monitoring, and file change monitoring to detect ransomware and hook crypto API functions to retrieve the key. To test the feasibility of key extraction from RøB, we created a Node.js script utilizing puppeteer [53] to periodically capture heap snapshots of the web application. We performed two experiments. First, we ran RøB on a test directory and retrieved two heap snapshots: one during the attack and another afterward. Second, we adjusted our script to continuously capture heap snapshots of RøB every 5 seconds, retrieving a total of four different snapshot files. We inspected all output files to search for our predefined key. While we did not encounter the key in the files from our first experiment, we detected the key in a single

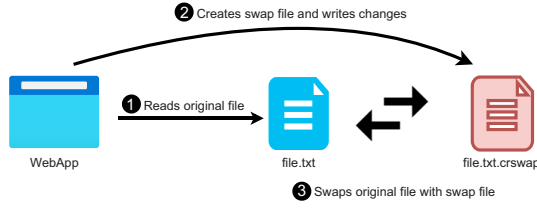


Fig. 2. The file change behavior of the file system access API.

file from our second experiment. The focus of these experiments was the potential extraction of a raw key during a browser-based ransomware attack. Nevertheless, if the intermediate key representations (e.g., AES T-tables) are detected in the snapshot, it would also be sufficient to enable the recovery of the key as well. Our experiments reveal that extracting the key during a browser-based ransomware attack is feasible, but it is not practical. Firstly, taking heap snapshots (each snapshot is $\approx 4.8\text{MB}$) of every website the user is visiting and storing them for further analysis requires a huge memory and may potentially affect the user experience. Additionally, RøB can solely utilize the RSA public key encryption to encrypt each file, potentially evading this type of defense solution.

6 RøBGUARD: A Novel Defense Solution

In this section, we give the methodology of RøBGUARD and propose a novel lightweight defense system called RøBGUARD that is capable of detecting RøB-like attacks effectively.

6.1 Methodologies of RøBGUARD

In this subsection, we first present the key methodology utilized by RøBGUARD in stopping RøB-like ransomware attack. Afterward, we introduce two key indicators employed by RøBGUARD to identify malicious modification to a file.

6.1.1 Stopping the Attack. To stop the activity of RøB attack, a tailored behavioral analysis should be performed on RøB and its interactions with the victim files should be analyzed. To achieve that, we monitor the file system events of RøB during the attack by instrumenting `inotifywait`. We illustrate the file system activities of the RøB in Figure 2. Specifically, ❶ it first reads all of the files in the accessed directory ❷. After that, it creates swap files for each file in the directory and writes the modifications to the swap files. The API names the swap files by appending the “.crswap” extension to the original files. Lastly, as represented in ❸ it swaps the content of the original files with the swap files. Note that, the modifications made by RøB are not permanent until step ❸. Therefore, intercepting the activity of a web application in this step prevents RøB from making permanent changes to the original files and stops the attack. On the other hand, since this distinct behavior is related to the File System Access API, this would also stop the activity of the benign application. Thus, *to detect a RøB-like browser-based ransomware attack, the intent of the modification made by a web application to a user’s files must be identified*. To achieve that, the original files and their modified versions should be analyzed. These two versions of files are obtainable at the same time by pausing the activity of the web application in step ❸ in Figure 2.

6.1.2 Identifying Malicious Modification. To identify the intent of the modification (i.e., ransomware or benign), we identify two key indicators: *Entropy Change* and *Size Change* between the original and modified files.

Entropy Change. The entropy of a file represents the amount of randomness of the file content. It is calculated by utilizing Shannon Entropy algorithm that returns a value between 1-8 for a file,

where 1 is low entropy and 8 is high. Since files with high entropy values could indicate the file is encrypted, many ransomware defense solutions used the entropy of a file as an indicator to identify encrypted files [31, 54, 55]. Particularly, they define a specific threshold value and mark the file as encrypted if the measured entropy of a file is above the calculated threshold. On the other hand, comparing the file entropy value with the threshold value may cause the system to identify file types that naturally have high entropy values (i.e., PDF, xlsx) as encrypted. So, these types of solutions may introduce high false positive cases. Nevertheless, instead of comparing the entropy of a file with the threshold value, the entropy change between the files after the modification can be used to remedy the drawbacks of such defense solutions.

Size Change. We determine that the size change between the original file and its modified version can be used as a second indicator to detect the malicious/benign intent of the modification. Since the encryption operation does not expand the data included in the file, the size of the file remains relatively the same after the encryption. On the other hand, benignly modifying the file (i.e., adding, extracting data, converting the file type) changes the size of the file. Therefore, the size and entropy differences between the original and its modified version would be an efficient indicator to identify whether the files are encrypted or benignly modified.

Thus, to identify the intent of the modification, size, and entropy change between the original file and its modified version should be determined as indicators. After that, these indicators are used as input to machine learning classifiers trained with a dataset that includes file and size differences between the original files and their benignly modified versions, and size and entropy differences between the original files and their encrypted versions. If the classifier identifies the modification as malicious activity the web application is terminated.

6.2 RØBGUARD Framework

RØBGUARD is a dynamic defense system that aims to detect browser-based ransomware attacks effectively. As shown in Figure 3, RØBGUARD has four main components. While the user is browsing on the web, the Hooking module monitors the web applications that use the FSA API via hooking. When such a web application is detected, the module stops the application's activity before it makes persistent changes on the user files and forwards the path of the directory that has been accessed by the web application to the Preprocessor module. The Preprocessor module calculates the size and entropy differences between files and their associated swap files and passes the calculated values to the File Classifier module, which classifies the modifications made to the files as benign or malicious (ransomware). Finally, based on the classification result, the Notifier module either authorizes the browser process activity or alerts the user about the malicious activity and kills the browser process.

6.3 Implementation of RØBGUARD

We implemented a prototype version of RØBGUARD as an application for Chrome. Our implementation was in Python 3.0 with additional scripts in Node.js. As the FSA API is available only in Chrome, Edge, and partially in Opera, which are all Chromium-based browsers, Chrome was the optimal choice for us. However, we do not expect significant implementation changes for Edge. Our implementation was performed and evaluated on a system running macOS Big Sur 11.0.1 with a 2,7 GHz Dual-Core Intel Core i5 processor and 8GB of available RAM. We conducted all of our experiments on Google Chrome 85. Technical details regarding the implementation of each component of RØBGUARD are as follows.

We note that we implemented RØBGUARD as a research prototype; therefore, its application in real-world scenarios may reveal limitations that were not apparent during initial testing. This prototype serves primarily to demonstrate the feasibility and potential impact of our proposed

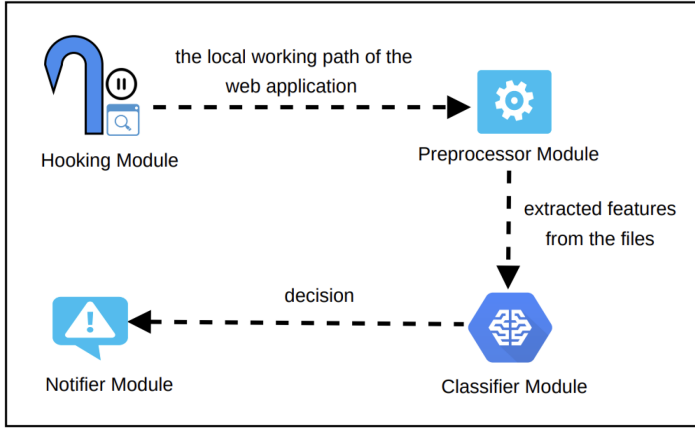


Fig. 3. Components of RøBGUARD.

solutions under controlled conditions. Future work will be required to refine the approach, enhance its robustness, and validate its efficacy in more varied and dynamic environments.

6.3.1 Hooking Module. We implemented the Hooking module as a script written in node.js. This script utilizes Puppeteer [53], a Node library that is able to interact with Google Chrome over DevTools Protocol. The script wraps functions of the FSA API such as `ShowDirectoryPicker`, `removeEntry`, `write`, to fully control and prevent the possible malicious persistent changes on the user files. Specifically, whenever a web application calls the `removeEntry` function of the API, this hooking logic is activated which prevents the original `removeEntry` function from operating and inserts the logic of the Hooking module in between. We implemented the pausing activity of this module on web applications via disabling JavaScript of the web application via `page.setJavaScriptEnabled(false)` function of puppeteer which pauses the JavaScript activity of only the web application.

6.3.2 Preprocessor Module. The Preprocessor module gets the path and the names of the original and the swap files from the Hooking module and retrieves the files. Subsequently, it calculates the size and entropy differences between the original and the swap files. After that, the calculated values are fed to the File Classifier module. We implemented the Preprocessor module as a Python 3 script which calculates the size of the files using `os.path.getsize` Python function, and the entropy values of the files via an entropy function that calculates the Shannon Entropy of a file.

6.3.3 File System Access Module. RøB reads a file, encrypts it, and overwrites the original file with the encrypted contents as outlined in Algorithm 1. In Line 1, `showOpenFilePicker()` function is invoked which opens a directory select dialogue for the user. When the user selects a directory and grants a read permission, the API returns a `FileSystemDirectoryHandle` that contains the methods needed to interact with the files of the user. The statements in Lines 2-8 iterate through files in the directory selected by the user. `getFile()` method called in Line 3 returns a `FileSystemEntry` object of a file and forwards it to `encrypt()`, which returns the encrypted file contents. Afterward, `createWritable()` in Line 5, asks for the user's permission to modify the file. Once the permission to write has been granted by a user, the `write()` method overwrites the file. Since the API does not need users to grant explicit permissions for each file within the directory (as read and write permissions are obtained for the directory), RøB can continue performing its malicious actions without prompting the users as long as the session continues.

ALGORITHM 2: Classification Retriever

```

1 fileValues = Preprocessor(path);
2 results = [];
3 for file to fileValues do
4   | results.append(model.predict(fileValues));
5 end

```

6.3.4 File Classifier Module. The File Classifier module gets inputs from the Preprocessor module and outputs whether the modifications made by the web application on the files are malicious (ransomware) or benign. Algorithm 2 outlines the steps followed by the File Classifier module. Specifically, this function gets the measured size and entropy differences of each file from the Preprocessor module. In Line 2, the variable `results` is declared, while will store the results of the classification of each file. In Line 4, the the `model.predict()` scikit-learn function is used to classify each file. The function returns an integer representing whether the file is benignly modified (0) or encrypted (1). Each classification result is appended to `results` list that the notifier will use to perform its task. The file classifier is built using the scikit-learn library and trained with a dataset that includes size and entropy differences between original files and their artificially modified versions and encrypted versions for various file types. The details regarding the training dataset are discussed in Section 7. RØBGUARD aims to detect RØB-like ransomware attacks effectively. Based on these needs, we have selected four supervised machine learning algorithms such as **Random Forest (RF)**, **K-Nearest Neighbor (KNN)**, **Decision Tree (DT)**, and XGBoost, and discuss their efficacy.

6.3.5 Notifier Module. We implemented the Notifier module as a script. Based on the results of the Classifier module, this module either allows the web application to continue to run by re-enabling JavaScript (i.e., `page.setJavaScriptEnabled(true)`) for that web application or it displays a notification message to the user warning that ransomware is detected.

7 Evaluation of RØBGUARD

We evaluate the efficacy of RØBGUARD in detecting RØB-like attacks by considering the following research questions:

- **RQ1** What is the accuracy of RØBGUARD in detecting RØB-like attacks and differentiating benign file modifications on different file types from the malicious (ransomware) ones? (Section 7.2).
- **RQ2** What is the overhead of the RØBGUARD? (Section 7.3.)

7.1 Training Environment and Methodology

Dataset Creation and Feature Extraction. In order to train and evaluate the performance of RØBGUARD, we needed to have a comprehensive dataset that covers different types of files with their various versions where users perform diverse changes/modifications. We searched for such databases in various resources including IEEE DataPort, Google Dataset Search, Kaggle, government websites (Data.gov and open data websites of various states and countries), and UCI Machine Learning Repository. However, we could not find a suitable dataset for our needs. For this reason, we decided to create our own dataset that covers different file types with various realistically generated versions. We provided the breakdown of our dataset in Table 2.

We selected five file types, namely pdf, docx, xlsx, txt, jpeg, that may contain sensitive data of users and enterprises. For example, xlsx, PDF, and docx are the file types that are

Table 2. Dataset Breakdown (Number of Files)

	txt	xlsx	jpeg	docx	pdf	Total
Benign	100K	100K	100K	100K	100K	500K
Malicious (Encrypted)	100K	100K	100K	100K	100K	500K

frequently used by enterprises to store and process their business data. To form our dataset, we collected files for each file type from the Digital Corpora. The Digital Corpora consists of almost 1 million real files from various file types that were crawled by Garfinkel et al. [22] from government websites and distributed freely. Using the Digital Corpora, we collected 1,000 files from each file type. The average file size of each type of a file is 0.67MB for pdf, 0.24MB for docx, 0.29MB for xlsx, 0.58MB for txt, and 0.15MB for jpeg formats.

To mimic user behavior on the files, we considered adding and removing contents from the files to generate different versions. While doing that, we paid attention to preserving the file formats. To have a comprehensive dataset, we created 100 different versions of each file. The first 50 versions are content removed versions of files are created to mimic deletion operations made by users. On the other hand, the other second 50 versions are content-added versions of the files are created to reflect data appending operation made by the user. We developed Python scripts that perform these procedures on the files. The low-level details of how we created 100 different versions of each file with respect to each file type are summarized as follows:

Modification of txt files For the content removal operation from a text file, we define a range between 1 and n , where n is the number of words in the text file. Afterward, we generate a random number r in the range of $[1, n]$ using Python's built-in random function that generates random numbers with respect to Uniform Random Distribution. Then, starting from the end of the text file, we remove r words from the original version of the text file and created the new version of the file. For the content insertion operation, we generate another random number r in the same way and we randomly choose r words from the word database that includes the contents of all the text files in our dataset and append these randomly chosen words to the end of the text file.

Modification of docx files For the content removal operation from a docx file, we used a similar methodology with text files. Specifically, we first generate a random number r between 1 and n where n is the number of words in the docx file. Then, starting from the end of the docx file, we remove r words from the docx file to create the content removed version. In the context of creating content-added versions of the docx file, we utilized our docx content database. Such that, we retrieve r words randomly from the docx content database and also randomly choose a jpeg file from our jpeg file database and append them to the docx file.

Modification of pdf files For the content removal operation from a pdf file, we first generate a random number r that is in the range of $[1, n]$ where n represents the number of pages of the pdf file. Afterward, we remove the content that resides on the r th page of the pdf file to generate the content-removed version of the file. To generate the content-added version of the file, we randomly choose a pdf file from our dataset and add its content on the r th page to the end of the pdf file.

Modification of jpeg files We perform the content-removal operation on a jpeg file by cropping the file randomly. To achieve this, we define two random variables, namely r_1 that is between 1 and n , and r_2 that is between 1 and m , where n represents the width and m represents the height of the jpeg file. By using these randomly generated width and height values, we crop the jpeg file starting from the left top corner ((0,0) coordinates). After this operation, the jpeg file is cropped to become an $r_1 \times r_2$ image. We create the other 50 different versions of a jpeg file by merging it with another jpeg file randomly selected from our database, which includes all jpeg files in our dataset.

Modification of `xlsx` files To perform the removal operation on an `xlsx` file, we first calculate the number of rows in an `xlsx` file. Then, we define a range between 1 and n , where n is the number of rows in the `xlsx` file. Afterward, we create a random number r in this range and remove r rows from the end of the `xlsx` file. To perform adding operation, we add a random number (r) of rows to the end of the `xlsx` file that is retrieved from `xlsx` file database.

Reflecting Malicious Changes. To reflect malicious changes (i.e., encryption with RøB) on the files, we encrypted each file including modified versions in our dataset with RøB that uses the AES-256 encryption algorithm.

A breakdown of our dataset including benign (original+modified) and encrypted versions of files is outlined in Table 2. In total, we have 1 million files (500K benign and 500K encrypted files) in our dataset.

Classification Accuracy Metrics. To evaluate RøBGUARD, we utilized the following performance metrics: Recall, F1-score, Precision (Prec.), and Accuracy (Acc.). We also report the True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN) cases, and the ROC curve (true positive rate against false positive rate). TP indicates the number of benign modifications that are detected correctly. TN refers to the number of correctly identified malicious modifications. FP states malicious modifications that are classified as benign modifications. Finally, FN defines the number of benign modifications that are classified as malicious.

Reproducibility. To make our evaluation and experiments reproducible, we trained the File Classifier module with the dataset described. Specifically, the size and entropy differences between the original files and their versions are calculated and labeled as benign modifications and their encrypted versions are calculated and labeled as malicious. To prevent overfitting, we utilized a 10-fold cross-validation approach that trains the classifier iteratively on 90% of the training data and tests on the remaining data 10%. The final results are obtained after 10 iterations by measuring the average values of all models. This way, the error rate of the predictive model is minimized. To determine the best classifier, we measure the performance metrics for various classifiers that were previously used in the various malware detection systems [41, 47], particularly: Random Forest (RF), K-nearest Neighbor (KNN), Decision Tree (DT), and XGBoost.

7.2 Accuracy Analysis

In this subsection, we present the results for the evaluation of the efficacy of RøB in detecting RøB-like browser-based ransomware attacks and discriminating benign file modifications from malicious ones. Table 3 presents the performance evaluation results. We observe that the RF classifier outperforms other classifiers in the context of identifying encrypted `txt` and `docx` files by introducing only one FP case. In the case of `jpeg` files, the DT classifier presents the best performance by presenting 99.5% accuracy without introducing any FP. Finally, KNN achieves the best accuracy performance on `xlsx` files introducing only two FP cases and achieving 99% accuracy. Our evaluations show that RøBGUARD can successfully detect RøB attacks while efficiently differentiating benign web applications from malicious ones. Therefore, the implementation includes different classifiers and a particular classifier can be chosen with respect to a specific file type. For example, in the case of `xlsx` files, the XGBoost classifier can be used to identify the intent of the modifications on the `xlsx` files.

7.3 Overhead Analysis

The estimated overhead introduced by the proof-of-concept-implementation of RøBGUARD is presented in Table 4. RøBGUARD aims to protect victims from RøB-like attacks. As explained in Section 4, the Hooking module operates in the background, interfacing directly with the FSA API

Table 3. Comparison of Machine Learning Models

Model	File Type	Accuracy	Recall	Precision	F1 Score	TP	TN	FN	FP
RF	TXT	0.99	0.99	0.99	0.99	99997	99999	3	1
	PDF	0.99	0.99	0.99	0.99	100000	99981	0	19
	JPEG	0.99	0.99	0.99	0.99	99996	99996	4	4
	DOCX	0.99	0.99	0.99	0.99	99997	99999	3	1
	XLSX	0.99	0.99	0.99	0.99	99938	99710	62	290
KNN	TXT	0.99	0.99	0.99	0.99	100000	99982	0	18
	PDF	0.99	0.99	0.99	0.99	99988	99990	12	10
	JPEG	0.99	0.99	0.99	0.99	99995	99989	5	11
	DOCX	0.99	100	0.99	0.99	100000	99982	0	18
	XLSX	0.99	0.99	0.99	0.99	99692	99888	308	112
DT	TXT	0.99	0.99	0.99	0.99	99998	99995	2	5
	PDF	0.99	0.98	0.99	0.99	99951	99981	49	19
	JPEG	0.99	0.99	0.99	0.99	99992	99992	8	8
	DOCX	0.99	0.99	0.99	0.99	99996	100000	4	0
	XLSX	0.99	0.99	0.99	0.99	99700	99942	300	58
XGB	TXT	0.99	0.98	0.99	0.99	99997	99991	3	9
	PDF	0.99	0.99	0.99	0.99	100000	99983	0	17
	JPEG	0.99	0.99	0.99	0.99	99995	99991	5	9
	DOCX	0.99	0.99	0.99	0.99	99998	99995	2	5
	XLSX	0.99	0.99	0.99	0.99	99710	99935	290	65

Table 4. Overhead of RøBGUARD for a 25MB File

Usage	Hooking	Preprocessor	Classifier	Notifier
RAM MB	Negligible	≈ 25	0.052	Negligible
CPU(%)	Negligible	4-5	1.3	Negligible
Time(s)	Negligible	1.74	0.003	Negligible

functions. We monitored the overhead of Chrome while the FSA functions were being hooked and observed that the Hooking module does not impose substantial overhead on the system. The overhead incurred by the Preprocessor module is proportional to the size of the processed file. In the worst case, this module calculates the entropy and size of a 25MB file in 1.74 seconds, utilizing only 25MB of RAM and consuming on average 4-5% of CPU. Furthermore, it processes a 50MB file in 3.30 seconds, using approximately 50MB of RAM and 4-5% CPU on average. Similarly, it requires approximately 100MB RAM and 4-5% CPU on average to process a 100MB file in 6.38 seconds. Regarding the Classifier module, since it relies on a pre-trained model, the overhead during model training is not a consideration. However, the overhead during prediction varies among machine learning classifiers. For instance, the RF classifier utilizes only 0.057MB RAM and 1.6% CPU to classify a file in 0.021 seconds, the KNN classifier requires 0.59MB RAM and 1.9% CPU to classify a file in 0.004 seconds, the DT classifier uses 1.3% CPU and 0.052MB RAM to classify a file in 0.003 seconds, and the **Extreme Gradient Boosting (XGB)** classifier utilizes 0.70MB RAM and 10% CPU to classify a file in 0.010 seconds.

As discussed in Section 6.3, RøBGUARD temporarily pauses the web application's activity only while classifying files as encrypted or not. Due to the efficiency of our machine learning model, which uses just two features, this pausing time is minimal. This brief interruption is proportional to the size of the file being processed. While the proof-of-concept implementation of RøBGUARD shows promise in detecting browser-based ransomware attacks, it is essential to acknowledge its limitations. As noted, the current implementation's memory usage is proportional to the file

size. Hence, processing larger files might cause a significant overhead to the user system without additional memory management and processing techniques.

8 Discussion

The performance evaluation and overhead analysis of RøBGUARD given in Section 7 demonstrate that RøBGUARD is highly effective while being fast, practical, and computationally inexpensive. In this section, we discuss the benefits of RøBGUARD and compare it with the current ransomware defense solutions.

False Positive Cases. Our performance evaluations demonstrate a promisingly low rate of false positives across various file types, with exceptionally low rates for TXT and DOCX files at $\approx 0.00001\%$, JPEG files at $\approx 0.00004\%$, and slightly higher rates for PDF and XLSX files at $\approx 0.00019\%$ and $\approx 0.00290\%$, respectively. While these results highlight the effectiveness of our current machine learning models in accurately distinguishing between benign and malicious file modifications, there may still be room for improvement, particularly for file types like XLSX where the FP rate, though low, could be further minimized. In response to the need for enhanced accuracy, one promising approach may involve the continuous training of RøBGUARD's detection models based on individual user behavior concerning file modifications. This adaptive learning strategy would allow RøBGUARD to dynamically adjust its detection algorithms based on real-time feedback and evolving patterns of how files are typically modified within the user's environment. By integrating this level of behavioral analysis, the system can become more attuned to the nuances of each user's interaction with files, potentially reducing the likelihood of misclassifying benign activities as threats. This continual learning process may not only enhance the specificity of our detection capabilities, but also ensure that RøBGUARD remains robust against the diverse and changing tactics employed by malware, thereby improving its overall reliability and reducing disruptions caused by false alarms.

Extendable File Classifier Module and Platform Independence. Our extensive performance evaluation shows that the File Classifier module of RøBGUARD classifies the intent of the modification by requiring only two features. Even though our dataset includes five file types, other file types could be added to the training dataset and the File Classifier module, thus RøBGUARD, could classify the intent of modifications on the other file types as well. Moreover RøBGUARD does not utilize browser-specific or operating system-specific features/tools. The tools that we instrument to implement RøBGUARD in Linux are also available in the other operating systems. For instance, the Monitoring module can be implemented with a FileSystemWatcher in Windows and FSWatch in MacOS. Furthermore, the process of the web applications can be controlled by the targeted operating systems' shell environments.

Freedom from Administrative Privileges. Ransomware defense solutions can require administrative privileges to monitor the system events or network traffic. However, this requirement introduces issues for the end-users who do not have administrative rights. Moreover, such defense solutions that require administrative privileges may result in additional security and privacy issues since they have a right to monitor the activities of every application that is running on the host system. RøBGUARD does not obligate end-users to have administrative rights as long as the browser process is created in the user level. Thus, users do not need to worry about additional security issues.

Resilience against Common Evasion Attempts. Like any other malware, ransomware strains are developing several methods to evade detection systems. For example, they use code obfuscation techniques to bypass static analysis-based methods or implement their own encryption algorithms to evade API-call based detection systems. Although previously developed ransomware detection

systems might be vulnerable to such evasion attempts, RØBGUARD stands resilient against these attempts since RØBGUARD does not rely on static analysis features, API-calls, or easy to obfuscate network features (e.g., domain names, hard-coded IP addresses, etc.).

9 Future Research Directions

In this section, we outline potential directions for future research, aimed at advancing the development of security solutions that are not only effective but also practical, adaptable, and user-centric. As browser-based ransomware continues to evolve in complexity and stealth, it is crucial that research also expands into new areas that can address emerging challenges and improve existing methodologies.

Investigating Efficiency of Browser-based Ransomware Attacks. Given the evolving landscape of ransomware attacks, it becomes crucial to reassess assumptions about the relative ease and viability of web-based ransomware compared to traditional methods. While browser-based ransomware attacks do not require the victim to download and install any applications, which might suggest simplicity, this perspective may not fully capture the complexities involved in achieving a high conversion rate that makes such attacks economically viable. Therefore, we suggest that the academic and web communities investigate the comparative effort and efficacy of browser-based versus traditional ransomware attacks. Furthermore, to better articulate the impact of browser-based ransomware it is crucial to investigate the temporal aspects that could either restrict or enable the damage potential of these attacks. Unlike traditional ransomware, which typically has ongoing access to the system until detected and removed, browser-based ransomware operates under the constraint that it can only execute while its tab is active. This fundamental difference raises several pertinent questions about the feasibility and strategy behind such attacks. By analyzing these aspects future research can significantly advance our understanding of browser-based ransomware's capabilities, limitations, and the strategic considerations attackers must account for.

Usage of Deep Learning. As RØBGUARD has been implemented to use in real-world web application, we aim to make RØBGUARD lightweight. So, our decision to utilize machine learning classifiers rather than deep learning models was driven primarily by considerations of computational efficiency and the operational context of RØBGUARD. Deep learning models, while potent in improving classification accuracy, generally require substantial computational resources that may not align with the lightweight and real-time requirements of RØBGUARD. For instance, training deep learning models typically involves processing large datasets with complex network architectures that consist of multiple hidden layers. This not only increases the requirement for high computational power but also necessitates significant memory usage. To illustrate, a standard deep learning model like a **Convolutional Neural Network (CNN)** for file classification could require gigabytes of RAM and substantial GPU resources for efficient training and inference. Training such models often involve backpropagation algorithms that adjust thousands, if not millions, of parameters, which can be computationally intensive and time-consuming. For instance, training a simple CNN on a moderate dataset can utilize up to 11GB of GPU memory and several hours of processing time, even on high-performance hardware. In contrast, the machine learning classifiers used by RØBGUARD such as Random Forest (RF) and Decision Trees (DT), are significantly more resource-efficient. This allows RØBGUARD to deliver the real-time performance necessary for operational efficacy without compromising the system's responsiveness and usability. However, given the significant impact of the browser-based ransomware attacks, exploring the integration of deep learning methods in future iterations of our system could be highly beneficial. Although our current approach prioritizes computational efficiency to ensure real-time performance, the

evolving threat landscape might justify revisiting this trade-off. Future work could focus on developing optimized deep learning models that balance enhanced classification accuracy with feasible computational demands. Such research could involve lightweight neural network architectures or the application of transfer learning techniques, which could potentially offer a promising compromise by leveraging pre-trained models adapted to our specific context with reduced resource requirements.

Utilizing of Physical Side Channels. Side channels are important techniques to detect malware and anomaly in the system. Previously this technique has demonstrated the effectiveness of using these physical indicators to detect IoT malware [19, 38], cryptomining [21, 37], hardware trojans [50, 51], and fingerprinting [42]. Therefore, future research could focus on developing methodologies to capture and analyze these physical signals specifically in the context of browser environments. For example, an unexpected spike in power usage or anomalous electromagnetic emissions during a browsing session could signal the execution of ransomware scripts or unauthorized encryption processes. By monitoring these physical indicators, the researchers develop a non-intrusive, real-time detection system that complements our existing software-based defenses. By integrating multi-modal side channel data with our existing software-based detection frameworks, researchers potentially identify attack patterns that are otherwise difficult to detect through conventional methods alone. Such a comprehensive approach would not only enhance the detection capabilities of our systems but also make them more resilient to sophisticated evasion techniques employed by attackers.

Extension of Dataset and Real-world Experiments. In this work, we implemented RØBGUARD as a research prototype and performed the evaluation by creating a synthetic dataset. Hence, to enhance the realism and predictive accuracy of RØBGUARD, future research should focus on expanding the variety of operations included in our dataset. This expansion would involve incorporating file modifications that maintain file size while altering other properties like entropy. For example, operations could include mirroring a JPEG image, retouching a photo without changing its size, compressing a PDF file where minimal size reduction is achievable, and making modifications within a ZIP archive that do not affect its overall size. Additionally, assessing the false positive rate through more lifelike benchmarks or during actual usage will provide crucial insights into the practicality of our proposed solution. By integrating these types of file modifications and conducting real-world testing, we aim to refine RØBGUARD to better differentiate between benign and malicious activities across various file types and usage scenarios in detecting browser-based ransomware attacks.

Analyzing User Behavior. Another critical area for exploration is the interplay between user behavior and the effectiveness of security mechanisms against browser-based ransomware. In this context, future research could delve into how users interact with and respond to security alerts, with a focus on the cognitive and behavioral aspects influencing their decision-making processes [11, 14, 15]. Investigating user awareness and how promptly they respond to ransomware threats in real-time browsing scenarios can highlight gaps in current security protocols. Additionally, this research could assess the impact of educational interventions on improving user behavior and reducing susceptibility to attacks. By studying the influence of user interface design on user actions during ransomware alerts, researchers can identify key factors that either facilitate or hinder effective user interaction with security tools. The ultimate goal of this research would be to inform the development of more intuitive and proactive browser security solutions that align with natural user behaviors and cognitive patterns, thereby enhancing the overall efficacy of ransomware mitigation strategies in dynamic online environments.

10 Related Work

In this section, we articulate the related work and highlight their characteristics and differences from our work.

Ransomware Defense Research. The ransomware detection approaches for PCs can be grouped into two categories: static analysis-based and dynamic analysis-based solutions. Static analysis-based solutions [35, 64] use structural features such as strings and opcodes to detect ransomware. Dynamic analysis-based solutions use behavioral features such as network and registry activity, API/system call usage [4, 6, 30, 34, 65], I/O access patterns, and file system activity [47]. Since ransomware need communication with the C&C server for the attack, [5, 12, 13, 39] detect ransomware using network traffic features [5, 12, 13, 39], API/System calls are also used by several ransomware detection systems. In addition, registry changes [25, 26] and **DLL (Dynamic Link Library)** activity [52] are also used for ransomware detection. In [28], the authors designed an analysis environment where a suspected sample is fed to the system to analyze the behavior of the sample. Furthermore, the ransomware defense solutions of [29, 55] employ file system level features such as directory traversals and delete operation. Finally, some studies such as [36] use multiple behavioral features to detect ransomware.

Web API Security. Several works in the literature have analyzed the security and privacy of emerging web APIs. In [24], the authors identified a vulnerability in the Geolocation API, analyzed its impact, and discussed potential countermeasures. In [56], the authors explored a new attack vector through the screen-sharing API and discussed the effectiveness of the existing web defense systems. In [63], Weeks discussed the possible adverse effects of exploiting the FSA API, which includes data exfiltration and the potential code execution.

Differences from Existing Works. This work differs from the existing studies in several ways: (1) To the best of our knowledge, no study exists in the literature that demonstrates malicious use cases of the FSA API. In this study, we designed and implemented the first browser-based platform-independent ransomware attack. (2) We determined that the API does not limit malicious web applications that try to access and modify the files in external storage devices, data partitions, shared network folders, and cloud-integrated directories. (3) We determined that commercial antivirus solutions that promise ransomware detection via sensitive file monitoring and behavior monitoring cannot mitigate RøB. (4) We propose three different defense approaches to mitigate this new attack vector at different levels (i.e., browser, local, API). (5) We discuss the implications of RøB and present our recommendations. (4) In our work, we designed and implemented a novel platform-independent lightweight ransomware defense system that can effectively mitigate browser-based ransomware attacks. RøBGUARD detects RøB-like attacks in real-time with minimum performance overhead on the browsing experience of users. In addition, RøBGUARD builds upon a unique feature of the FSA API (i.e., creation of swap files) and stops RøB before permanent damage is made to files via function hooking which differentiate it from the existing defenses.

11 Conclusion

In this work, we designed and implemented the first browser-based ransomware - RøB. Our extensive evaluations with three different OSs, 29 distinct directories, and five cloud providers showed that RøB is capable of encrypting numerous types of files in various local directories, cloud-integrated directories, external storage devices, and network-shared folders. As existing ransomware detection systems including commercial antivirus solutions face several issues against RøB due to its distinct features, there was a need to propose a new defense solution against RøB-style attacks. Therefore, we proposed RøBGUARD, which uses function hooking and ML classification to detect RøB-based attacks. We designed, implemented, and analyzed the

performance of RøBGUARD. Our performance analysis of RøBGUARD show that it can detect RøB-like ransomware attacks efficiently. Finally, we outline future research directions to motivate further exploration in this domain.

Acknowledgments

We thank the anonymous reviewers for their helpful feedback and time.

References

- [1] 2022. The File System Access API with Origin Private File System. <https://webkit.org/blog/12257/the-file-system-access-api-with-origin-private-file-system/>
- [2] Abbas Acar, Güliz Seray Tuncay, Esteban Luques, Harun Oz, Ahmet Aris, and Selcuk Uluagac. 2024. 50 shades of support: A device-centric analysis of Android security updates. In *Network and Distributed System Security Symposium*.
- [3] Devdatta Akhawe and Adrienne Porter Felt. 2013. Alice in warningland: A large-scale field study of browser security warning effectiveness. In *22nd USENIX Security Symposium*.
- [4] B. Al-rimy, M. Maarof, Y. Prasetyo, M. Mohd Shaid Z. Syed, Shaid, and Asmawi Fadillah Mohd Ariffin. 2018. Zero-day aware decision fusion-based model for crypto-ransomware early detection. *International Journal of Integrated Engineering* (2018).
- [5] O. M. K. Alhawi, J. Baldwin, and A. Dehghantanha. 2018. Leveraging machine learning techniques for Windows ransomware network traffic detection. In *Cyber Threat Intelligence*.
- [6] Pieter Arntz. 2017. Analyzing Malware by API Calls. <https://blog.malwarebytes.com/threat-analysis/2017/10/analyzing-malware-by-api-calls/>
- [7] Avast. 2022. Avast One Essential Protection for Your Life Today. <https://www.avast.com/en-us/avast-one#pc>
- [8] AVG. 2022. Get Free Antivirus that's Trusted by Experts. <https://www.avg.com/en-us/homepage#pc>
- [9] Shrenik Bhansali, Ahmet Aris, Abbas Acar, Harun Oz, and A. Selcuk Uluagac. 2022. A first look at code obfuscation for WebAssembly. In *Proceedings of the 15th ACM Conference on Security and Privacy in Wireless and Mobile Networks*.
- [10] Brave. 2019. Remove Support for Native File System API. <https://github.com/brave/brave-browser/issues/11407#issuecomment-851742821>
- [11] Fernando Brito, Yassine Mekdad, Monique Ross, Mark A. Finlayson, and Selcuk Uluagac. 2025. Enhancing cybersecurity education with artificial intelligence content. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE TS'25)* (Pittsburgh, PA, USA). Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3641554.3701958>
- [12] Krzysztof Cabaj, Marcin Gregorczyk, and Wojciech Mazurczyk. 2018. Software-defined networking-based crypto ransomware detection using HTTP traffic characteristics. *Computers and Electrical Engineering* (2018).
- [13] Krzysztof Cabaj and Wojciech Mazurczyk. 2016. Using software-defined networking for ransomware mitigation: The case of CryptoWall. *IEEE Network* (2016).
- [14] Derin Cayir, Abbas Acar, Riccardo Lazzaretti, Marco Angelini, Mauro Conti, and Selcuk Uluagac. 2024. Augmenting security and privacy in the virtual realm: An analysis of extended reality devices. *IEEE Security & Privacy* 22, 1 (2024), 10–23. <https://doi.org/10.1109/MSEC.2023.3332004>
- [15] Derin Cayir, Reham Mohamed, Riccardo Lazzaretti, Marco Angelini, Abbas Acar, Mauro Conti, Z. Berkay Celik, and Selcuk Uluagac. 2025. Speak Up, I'm listening: Extracting speech from zero-permission VR sensors. In *NDSS*.
- [16] Yunang Chen, Yue Gao, Nick Ceccio, Rahul Chatterjee, Kassem Fawaz, and Earlene Fernandes. 2022. Experimental security analysis of the app model in business collaboration platforms. In *31st USENIX Security Symposium*.
- [17] Chromium. 2023. File System Access Web API - Chromium Security Model. <https://bit.ly/2kEi4nI>
- [18] A. Continella, A. Guagnelli, G. Zingaro, G. Pasquale, A. Barengi, S. Zanero, and F. Maggi. 2016. ShieldFS: A self-healing, ransomware-aware filesystem. In *Proceedings of the 32nd Annual Conference on Computer Security Applications*.
- [19] Fei Ding, Hongda Li, Feng Luo, Hongxin Hu, Long Cheng, Hai Xiao, and Rong Ge. 2020. DeepPower: Non-intrusive and deep learning-based detection of IoT malware using power side channels. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security* (Taipei, Taiwan) (ASIA CCS'20). Association for Computing Machinery, New York, NY, USA, 33–46. <https://doi.org/10.1145/3320269.3384727>
- [20] Dropbox. 2020. Version History Overview. <https://help.dropbox.com/files-folders/restore-delete/version-history-overview>
- [21] Ankit Gangwal and Mauro Conti. 2020. Cryptomining cannot change its spots: Detecting covert cryptomining using magnetic side-channel. *IEEE Transactions on Information Forensics and Security* 15 (2020), 1630–1639. <https://doi.org/10.1109/TIFS.2019.2945171>

- [22] Simson Garfinkel, Paul Farrell, Vassil Roussev, and George Dinolt. 2009. Bringing science to digital forensics with standardized forensic corpora. *Digital Investigation* (2009).
- [23] Google. 2023. View Activity & File Versions. <https://support.google.com/drive/answer/2409045>
- [24] Xiao Han, Junjie Xiong, Wenbo Shen, Zhuo Lu, and Yao Liu. 2022. Location heartbleeding: The rise of Wi-Fi spoofing attack via geolocation API. In *ACM SIGSAC Conference on Computer and Communications Security*.
- [25] Jinsoo Hwang, Jeankyung Kim, Seunghwan Lee, and Kichang Kim. 2020. Two-stage ransomware detection using dynamic analysis and machine learning techniques. *Wireless Personal Communications* (2020).
- [26] Brijesh Jethva, Issa Traoré, Asem Ghaleb, Karim Ganame, and Sherif Ahmed. 2019. Multilayer ransomware detection using grouped registry key operations, file entropy and file signature monitoring. *Journal of Computer Security* (2019).
- [27] Kaspersky. 2022. Kaspersky Total Security. <https://usa.kaspersky.com/total-security>
- [28] A. Kharaz, S. Arshad, C. Mulliner, W. Robertson, and E. Kirda. 2016. UNVEIL: A large-scale, automated approach to detecting ransomware. In *25th USENIX Security Symposium*.
- [29] A. Kharraz and E. Kirda. 2017. Redemption: Real-time protection against ransomware at end-hosts. In *Research in Attacks, Intrusions, and Defenses*. Springer International Publishing.
- [30] E. Kolodenker, W. Koch, G. Stringhini, and M. Egele. 2017. PayBreak: Defense against cryptographic ransomware. In *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*.
- [31] Kyungroul Lee, Sun-Young Lee, and Kangbin Yim. 2019. Machine learning based file entropy analysis for ransomware detection in backup systems. *IEEE Access* (2019).
- [32] Malwarebytes. 2023. Malwarebytes Browser Guard. <https://www.malwarebytes.com/browserguard>
- [33] Malwarebytes. 2023. Malwarebytes Premium for Windows. <https://www.malwarebytes.com/premium>
- [34] F. Martinelli, Francesco Mercaldo, C. Michailidou, and A. Saracino. 2018. Phylogenetic analysis for ransomware detection and classification into families. In *ICETE*.
- [35] M. Medhat, Samir Gaber, and Nashwa Abdelbaki. 2018. A new static-based framework for ransomware detection. *2018 IEEE 16th Intl. Conf. Congress* (2018).
- [36] S. Mehnaz, A. Mudgerikar, and E. Bertino. 2018. RWGuard: A real-time detection system against cryptographic ransomware. In *Research in Attacks, Intrusions, and Defenses*.
- [37] Yassine Mekdad, Giuseppe Bernieri, Mauro Conti, and Abdeslam El Fergougui. 2021. A threat model method for ICS malware: The TRISIS case. In *Proceedings of the 18th ACM International Conference on Computing Frontiers*. 221–228.
- [38] Yassine Mekdad, Faraz Naseem, Ahmet Aris, Harun Oz, Abbas Acar, Lenoardo Babun, Selcuk Uluagac, Güliz Seray Tuncay, and Nasir Ghani. 2024. On the robustness of image-based malware detection against adversarial attack. Springer.
- [39] J. Modi, I. Traore, A. Ghaleb, K. Ganame, and S. Ahmed. 2020. Detecting ransomware in encrypted web traffic. In *Foundations and Practice of Security*.
- [40] Mozilla. 2022. Mozilla Specification Positions. <https://mozilla.github.io/standards-positions/#file-system-access>
- [41] A. K. M. Newaz, Amit Kumar Sikder, Mohammad Ashiqur Rahman, and A. Selcuk Uluagac. 2020. A survey on security and privacy issues in modern healthcare systems: Attacks and defenses. *arXiv preprint arXiv:2005.07359* (2020).
- [42] Tao Ni, Xiaokuan Zhang, and Qingchuan Zhao. 2023. Recovering fingerprints from in-display fingerprint sensors via electromagnetic side channel. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (Copenhagen, Denmark) (CCS'23)*. Association for Computing Machinery, New York, NY, USA, 253–267. <https://doi.org/10.1145/3576915.3623153>
- [43] OneDrive. 2021. Restore a Previous Version of a File Stored in OneDrive. <https://support.microsoft.com/en-us/office/restore-a-previous-version-of-a-file-stored-in-onedrive-159cad6d-d76e-4981-88ef-de6e96c93893>
- [44] Ori Or-Meir, Nir Nissim, Yuval Elovici, and Lior Rokach. 2019. Dynamic malware analysis in the modern era—A state of the art survey. *ACM Comput. Surv.* (2019).
- [45] Harun Oz, Ahmet Aris, Abbas Acar, Güliz Seray Tuncay, Leonardo Babun, and Selcuk Uluagac. 2023. RøB: Ransomware over modern web browsers. In *32nd USENIX Security Symposium (USENIX Security'23)*. USENIX Association, Anaheim, CA, 7073–7090. <https://www.usenix.org/conference/usenixsecurity23/presentation/oz>
- [46] Harun Oz, Ahmet Aris, Abbas Acar, Güliz Seray Tuncay, Amin Kharraz, and Selcuk Uluagac. 2024. (In) security of file uploads in Node.js. In *Proceedings of the ACM Web Conference 2024 (WWW'24)*. Association for Computing Machinery, New York, NY, USA.
- [47] Harun Oz, Ahmet Aris, Albert Levi, and A. Selcuk Uluagac. 2022. A survey on ransomware: Evolution, taxonomy, and defense solutions. *ACM Comput. Surv.* 54, 11s, Article 238 (Sept. 2022), 37 pages. <https://doi.org/10.1145/3514229>
- [48] Harun Oz, Daniele Cono D'Elia, Güliz Seray Tuncay, Abbas Acar, Riccardo Lazzeretti, and Selcuk Uluagac. 2024. With great power comes great responsibility: Security and privacy issues of modern browser application programming interfaces. *IEEE Security & Privacy* (2024), 2–11. <https://doi.org/10.1109/MSEC.2024.3443625>
- [49] Harun Oz, Faraz Naseem, Ahmet Aris, Abbas Acar, Güliz Seray Tuncay, and A. Selcuk Uluagac. 2022. Poster: Feasibility of malware visualization techniques against adversarial machine learning attacks. In *43rd IEEE Symposium on Security and Privacy (S&P'22)*.

- [50] Harun Oz, Güeliz Seray Tuncay, Ahmet Aris, Amin Kharraz, and Sencuk Uluagac. 2024. Beyond the upload button: A 10-year retrospective on security issues within file upload. *IEEE Communications Magazine* (2024).
- [51] Hammond Pearce, Virinchi Roy Surabhi, Prashanth Krishnamurthy, Joshua Trujillo, Ramesh Karri, and Farshad Khorrami. 2022. Detecting hardware Trojans in PCBs using side channel loopbacks. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 30 (07 2022), 1–12. <https://doi.org/10.1109/TVLSI.2022.3171174>
- [52] S. Poudyal, K. P. Subedi, and D. Dasgupta. 2018. A framework for analyzing ransomware using machine learning. In *2018 IEEE Symposium Series on Computational Intelligence (SSCI'18)*. 1692–1699.
- [53] Puppeteer. 2023. Puppeteer. <https://pptr.dev/>
- [54] J. Sangmoon and W. Yoojae. 2018. Ransomware detection method based on context-aware entropy analysis. *Soft Computing* (2018).
- [55] N. Scaife, H. Carter, P. Traynor, and K. R. B. Butler. 2016. CryptoLock (and Drop It): Stopping ransomware attacks on user data. In *IEEE 36th International Conference on Distributed Computing Systems*.
- [56] Yuan Tian, Ying Chuan Liu, Amar Bhosale, Lin Shung Huang, Patrick Tague, and Collin Jackson. 2014. All your screens are belong to us: Attacks exploiting the HTML5 screen sharing API. In *IEEE Symposium on Security and Privacy*.
- [57] Trendmicro. 2022. Antivirus+ Security. https://www.trendmicro.com/en_me/forHome/products/antivirus-plus.html
- [58] Can I use. 2023. Can I use File System Access API? <https://caniuse.com/native-file-system-api>
- [59] Amber van der Heijden and Luca Allodi. 2019. Cognitive triaging of phishing attacks. In *28th USENIX Security Symposium*.
- [60] Emanuel von Zezschwitz, Serena Chen, and Emily Stark. 2022. “It builds trust with the customers” - Exploring user perceptions of the padlock Icon in browser UI (*IEEE Security and Privacy Workshops*). IEEE Computer Society.
- [61] W3C. 2023. File System Access. <https://wicg.github.io/file-system-access/#security-ransomware>
- [62] Web Incubator Community Group (WICG). 2023. File System Access. <https://wicg.github.io/file-system-access/>. Accessed: 2024-10-06.
- [63] Matt Weeks. 2021. Internal Affairs: Hacking File System Access from the Web. <https://i.blackhat.com/USA21/Wednesday-Handouts/us-21-Internal-Affairs-Hacking-File-System-Access-From-The-Web.pdf>
- [64] Bin Zhang, Wentao Xiao, Xi Xiao, Arun Kumar Sangaiah, Weizhe Zhang, and Jiajia Zhang. 2020. Ransomware classification using patch-based CNN and self-attention network on embedded N-grams of opcodes. *Future Generation Computer Systems* (2020).
- [65] J. Zhou, M. Hirose, Y. Kakizaki, and A. Inomata. 2020. Evaluation to classify ransomware variants based on correlations between APIs. In *6th International Conference on Information Systems Security and Privacy*.

Received 25 March 2024; revised 9 August 2024; accepted 8 September 2024